Engineering Task: From Requirements to UPPAAL

Real-Time Systems & HW/SW Co-Design — Tasks 1–6

Moshiour Rahman Prince

Course: Real-Time Systems & HW/SW Co-Design

May 11, 2026

Contents

1 Introduction **3**

2 Problem Understanding **3**
- 2.1 System Description . . . 3
- 2.2 Original High-Level Requirements (R1–R12) . . . 3
- 2.3 Engineering Challenges . . . 4

3 Prompting Strategy **4**
- 3.1 Prompting Principles Applied . . . 4
- 3.2 Critical Review Procedure . . . 4

4 Requirements Extraction **5**
- 4.1 Explicit System Assumptions . . . 5
- 4.2 Ambiguity Detection and Resolution . . . 5
- 4.3 Final SysML-Style Requirements . . . 5
- 4.4 Origin Classification . . . 5

5 Manual Requirements Validation and Traceability **6**
- 5.1 Per-Requirement Quality Check . . . 6
- 5.2 Conflict Scan . . . 6
- 5.3 Representation of Safety and Fairness . . . 9
- 5.4 Formal vs. Informal Discharge . . . 9
- 5.5 Traceability Table . . . 9

6 UPPAAL Model **9**
- 6.1 Architecture . . . 9
- 6.2 Global Declarations . . . 11
- 6.3 Worker Template . . . 11
- 6.4 Controller Template . . . 12
- 6.5 System Declaration . . . 12
- 6.6 Requirement Discharge Map . . . 13

7 Verification Results **13**
- 7.1 Query Set . . . 13
- 7.2 Verification Matrix . . . 13
- 7.3 Why Each Property Holds (selected) . . . 13
- 7.4 Fairness Discussion . . . 15

7.4.1 What R11 actually says . . . 15
7.4.2 What UPPAAL CTL can express . . . 15
7.4.3 Reachability is not the same as fairness . . . 15
7.4.4 Leadsto is closer but still weaker . . . 15
7.4.5 Standard workarounds for strong fairness . . . 15
7.4.6 Conclusion on fairness . . . 16

8 Iterations and Changes **16**
8.1 Iteration 0 — Naive Initial Sketch . . . 16
8.2 Iteration 1 — Atomic Dequeue . . . 16
8.3 Iteration 2 — Single-Location Controller . . . 16
8.4 Iteration 3 — Local, Resettable `wasReleased` . . . 16
8.5 Iteration 4 — Clock and Bounded Working . . . 17
8.6 Iteration 5 — Range-Typed `qindex_t` . . . 17
8.7 Summary . . . 17

9 FreeRTOS Implementation (Outlook) **17**
9.1 Mapping . . . 17
9.2 Pseudo-code Skeleton . . . 17
9.3 Open Implementation Questions . . . 19

10 Comparison and Critical Reflection **19**
10.1 Artefact Comparison . . . 19
10.2 Where the LLM Helped . . . 19
10.3 Where the LLM Failed or Introduced Risk . . . 20
10.4 Manual Engineering Decisions . . . 20
10.5 Verified vs. Argued Requirements . . . 20

1 Introduction

The course Real-Time Systems & HW/SW Co-Design aligned the basic concepts of real-time systems — tasks, scheduling, deadlines, formal verification — during the lecture phase. This assignment applies them in an end-to-end engineering workflow that begins with an informal description, proceeds through requirements engineering, formal modelling in UPPAAL, and verification, with optional implementation in FreeRTOS.

The assignment explicitly permits the use of Large Language Models (LLMs) as engineering assistants but not as authorities. The engineer remains responsible for the final result and must review, correct, justify and trace every change suggested by the LLM. This report is structured to make that engineering trail visible: each chapter that uses LLM output contains a critical analysis sub-section, and a traceability table marks every requirement and every model element as LLM-generated, manually-tightened, manually-added, or rejected.

Scope: This report covers Tasks 1–6 of the assignment in full: the requirements engineering, requirements validation, UPPAAL model concept and implementation, UPPAAL verification, and the iteration/stabilisation loop. Task 7 (FreeRTOS implementation) is sketched as an outlook in Section 9 but is not implemented in this submission.

2 Problem Understanding

2.1 System Description

A system consists of two worker tasks and one controller. Each worker requests permission to start work. The controller receives requests, stores the requesting worker ID in a bounded FIFO queue, selects the next worker according to FIFO order, removes it from the queue, and sends a task-specific release message. Only after receiving this release may the selected worker start working.

2.2 Original High-Level Requirements (R1–R12)

R1. The system shall contain exactly two worker tasks and one controller.

R2. A worker shall request permission before starting work.

R3. A worker shall wait until it receives a task-specific release from the controller.

R4. The controller shall store incoming requests in a bounded FIFO queue.

R5. The controller shall release workers according to FIFO order.

R6. The controller shall remove a worker request from the queue after selecting it.

R7. At most one worker shall be released by the controller at a time.

R8. The queue shall never overflow.

R9. The queue shall never underflow.

R10. The model shall be deadlock-free.

R11. Fairness: if a worker repeatedly requests work and the system continues to operate, this worker shall not be ignored forever.

R12. Safety: a worker shall never enter the working state without a prior release from the controller.

2.3 Engineering Challenges

The list R1–R12 is short, but it mixes structural, behavioural, safety, and liveness properties and contains several implicit assumptions. The most important challenges are:

1. The bound on the FIFO queue is unspecified. "Bounded" must be made concrete before any model can be built.
2. "Task-specific release" could mean a per-worker channel array or a single broadcast channel with a filter; only the former matches the binary-channel semantics of UPPAAL.
3. R6 says the controller removes the entry after selecting it, but an early implementation that splits selection and removal into separate transitions is prone to data races.
4. R11 (fairness) is a property of the LTL form $\Box\Diamond\varphi \Rightarrow \Box\Diamond\psi$, which UPPAAL CTL cannot express directly.

These challenges drive the engineering decisions documented in Sections 4–8.

3 Prompting Strategy

3.1 Prompting Principles Applied

Following the recommendations from the assignment and the literature on LLMs in requirements engineering, the prompts followed seven principles:

1. Provide context. The full problem statement and R1–R12 were included verbatim in the prompt.
2. Define the role. The LLM was instructed to act as "an expert Real-Time Systems Engineer and formal-verification specialist".
3. Specify the output format. The expected format was explicitly fixed as SysML-style textual requirements (The <subject> shall <action> [under <condition>]).
4. Request assumptions explicitly. The prompt asked the LLM to list every implicit assumption as a separate ASM- item.
5. Request traceability. The prompt demanded a traceability table with at least the columns ID, Requirement, Source, Change Rationale.
6. Ask for safety and fairness. The prompt explicitly asked the LLM to encode R11 (fairness) and R12 (safety) as separate, named requirements.
7. Ask for ambiguity detection. The LLM was asked to flag every ambiguous phrasing in R1–R12 and propose a resolution.

3.2 Critical Review Procedure

Each artefact returned by the LLM was processed in three steps:

1. Read for plausibility. Catch obvious errors (swapped IDs, missing edge cases) before any deeper analysis.
2. Verify against the source. Cross-check every generated requirement with R1–R12; flag any drift.
3. Audit for missing assumptions. For every requirement, list the implicit assumptions and check whether the LLM made them explicit. If not, add them as ASM- items.

The result of this procedure is the classification used in Section 4: LLM-generated kept verbatim in spirit, LLM-generated tightened manually, manually added, and rejected.

4 Requirements Extraction

4.1 Explicit System Assumptions

The first manual addition was a list of explicit assumptions. Without them several R-items cannot be discharged formally.

ID	Assumption	Justification
ASM-001	$N = 2$ worker instances.	Fixed by R1; needed for static array sizing of channels.
ASM-002	$QSIZE = 4$, $QSIZE \geq N$.	R4 says "bounded" but gives no value; the chosen value makes the overflow guard non-trivial yet keeps the state space small.
ASM-003	A worker holds at most one outstanding request at any time.	Implicit in the cycle Idle→Requesting→Waiting→Working; required so $count \leq N$ is provable.
ASM-004	Inter-task communication uses UPPAAL binary, blocking, atomic channels.	Required for the `request[id]!/?` and `release[id]!/?` synchronisation semantics.
ASM-005	The underlying scheduler is weakly fair.	Without it, R11 is unprovable for any solution.
ASM-006	A worker stays in Working for a non-zero, bounded amount of time.	Avoids Zeno behaviour and gives the model a real-time character.
ASM-007	Worker IDs are integers in $[0, N-1]$ and are statically known at instantiation.	Required for `chan request[N]` / `chan release[N]` indexing.

Table 1: Explicit system assumptions.

4.2 Ambiguity Detection and Resolution

Natural language requirements frequently contain implicit assumptions and loose phrasing that cannot be directly translated into a formal, verifiable timed automaton. To bridge this gap, the initial problem statement (R1–R12) was systematically analysed for structural and semantic ambiguities. Table 2 details the identified issues and the concrete engineering decisions made to resolve them prior to finalising the SysML requirements.

4.3 Final SysML-Style Requirements

Following the resolution of initial ambiguities and the establishment of explicit system assumptions, the original natural language problem statement was formalised into concrete, SysML-style requirements[cite: 10, 39]. Table 3 groups these finalised requirements by their primary concern—structural constraints, task behaviour, queue invariants, and system-wide properties—establishing a clear baseline for the UPPAAL implementation and verification phases.

4.4 Origin Classification

To ensure full transparency and adhere to the principle of using the LLM as an assistant rather than an absolute authority, every requirement must be traced back to its conceptual origin. Table 4 categorises the final requirement set into four distinct classes: those retained verbatim in spirit from the LLM's initial output, those requiring manual tightening to become operational or verifiable, those added entirely by the engineer to close logical gaps, and those that were rejected. This classification highlights the necessary extent of manual engineering oversight.

#	Original phrasing	Ambiguity	Resolution
A1	R4: "bounded FIFO queue"	Bound is unspecified.	$QSIZE = 4$, encoded as a typedef range, not just as a guard.
A2	R5: "release workers according to FIFO order"	Order of *what*?	Order is defined w.r.t. enqueue order.
A3	R3/R7: "task-specific release"	Per-task channel or broadcast filter?	Per-task channel array `release[N]`.
A4	R6: "removes it after selecting"	"After" is ambiguous w.r.t. release send.	Atomic dequeue with release: removal and release on the same UPPAAL transition.
A5	R7: "at most one worker released at a time"	Concurrency unit unclear.	At most one `release[i]!` synchronisation per discrete UPPAAL step (binary-channel semantics).
A6	R11: "repeatedly requests" / "not ignored forever"	Vague liveness.	Re-stated as strong fairness $\Box\Diamond Requesting_i \Rightarrow \Box\Diamond Working_i$. UPPAAL limitation acknowledged.
A7	R12: "without a prior release"	"Prior" could mean any release ever.	Re-stated as: the most recent `release[i]` in the current request/work cycle (witnessed by `wasReleased`).
A8	R10: "deadlock-free"	Model only or implementation also?	Restricted to the UPPAAL model in this submission.

Table 2: Ambiguities detected in R1–R12 and the engineering resolution.

5 Manual Requirements Validation and Traceability

5.1 Per-Requirement Quality Check

A requirement is clear if a reader cannot reasonably misinterpret it, testable if a finite verification or test procedure can return pass/fail, and necessary if removing it would invalidate another requirement or the system goal.

5.2 Conflict Scan

A pair-wise scan was performed; the apparent conflicts and their resolutions are:

REQ-WRK-003 vs REQ-Q-001: with one outstanding request per worker, $count \leq N = 2$ never triggers $count = QSIZE = 4$, so the overflow guard becomes dead code. Not a real conflict: the guard is defensive — it makes the queue model independent of N and proves R8 as a property of the queue alone.

REQ-CTL-003 vs REQ-CTL-004: atomic dequeue+release vs. "at most one release at a time". No conflict; atomicity is even stronger — it implies single-release.

REQ-SYS-005 vs ASM-005: fairness is both a property and an assumption. Resolved by stating: REQ-SYS-005 is the property the system must achieve under ASM-005.

REQ-WRK-002 vs REQ-WRK-004: "wait until release" vs. bounded time. No conflict: timing is bounded only inside Working; Waiting has no invariant.

No genuine conflicts remain.

Concern	ID	Description	Source / Notes
Structural	REQ-SYS-001	The system shall consist of exactly two instances of the Worker task and one instance of the Controller task.	R1
Structural	REQ-SYS-002	Each Worker instance shall be identified by a unique ID $i \in [0, N-1]$, where $N = 2$.	Manually added; required for channel-array indexing.
Worker behaviour	REQ-WRK-001	A Worker shall send a `request[id]!` synchronisation before transitioning into its Working location.	R2
Worker behaviour	REQ-WRK-002	After issuing `request[id]!`, a Worker shall remain in Waiting until it has received exactly one `release[id]?` synchronisation.	R3, A3
Worker behaviour	REQ-WRK-003	A Worker shall complete the cycle Idle $\rightarrow$ Requesting $\rightarrow$ Waiting $\rightarrow$ Working $\rightarrow$ Idle in this order, holding at most one outstanding request at any time.	Manually added; encodes ASM-003.
Worker behaviour	REQ-WRK-004	A Worker shall stay in Working for at least 2 and at most 10 time units, measured by a local clock.	Manually added; encodes ASM-006.
Controller behaviour	REQ-CTL-001	On every `request[i]?` synchronisation, the Controller shall enqueue i at the queue tail and increment count, atomically with the synchronisation.	R4
Controller behaviour	REQ-CTL-002	When the queue is non-empty, the Controller shall release `queue[head]` (and only that ID) before any later-enqueued ID.	R5, A2
Controller behaviour	REQ-CTL-003	The dequeue (advance head, decrement count) and the `release[queue[head]]!` synchronisation shall happen on the same UPPAAL transition.	R6, A4
Controller behaviour	REQ-CTL-004	At most one `release[*]!` synchronisation per discrete step.	R7 (via ASM-004)
Queue invariants	REQ-Q-001	$0 \leq count \leq QSIZE$ shall be invariant.	R8, R9
Queue invariants	REQ-Q-002	The Controller shall not enqueue when $count = QSIZE$.	R8
Queue invariants	REQ-Q-003	The Controller shall not dequeue when $count = 0$.	R9
System-wide properties	REQ-SYS-003	The product automaton shall satisfy `A[] not deadlock`.	R10
System-wide properties	REQ-SYS-004	In every reachable state, $\text{Working}_i \Rightarrow \text{wasReleased}_i$	R12, A7

Table 3: Final SysML-Style Requirements

Class	IDs
LLM-generated, kept verbatim in spirit (translated to SysML form)	REQ-SYS-001, REQ-WRK-001, REQ-WRK-002, REQ-CTL-001, REQ-CTL-002, REQ-CTL-004, REQ-Q-001, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005
LLM-generated, manually tightened (made operational/verifiable)	REQ-CTL-003, REQ-Q-002, REQ-Q-003
Manually added to close gaps	REQ-SYS-002, REQ-WRK-003, REQ-WRK-004, all ASM-001..007
Rejected / collapsed	none

Table 4: Origin classification of the final requirement set.

ID	Clear?	Testable?	Necessary?	Comment
REQ-SYS-001	yes	yes	yes	Structural invariant.
REQ-SYS-002	yes	yes	yes	Required for channel-array indexing.
REQ-WRK-001	yes	yes	yes	Triggers controller participation.
REQ-WRK-002	yes	yes	yes	Forces synchronisation point.
REQ-WRK-003	yes	yes	yes	Makes $count \leq N$ provable.
REQ-WRK-004	yes	yes	yes	Real-time character; avoids Zeno.
REQ-CTL-001	yes	yes	yes	Defines storage behaviour.
REQ-CTL-002	yes	yes	yes	Defines selection rule.
REQ-CTL-003	yes	yes	yes	Prevents dequeue/release race.
REQ-CTL-004	yes	yes	partly	Automatic from ASM-004.
REQ-Q-001	yes	yes	yes	Composite invariant.
REQ-Q-002	yes	yes	yes	Direct UPPAAL query.
REQ-Q-003	yes	yes	yes	Direct UPPAAL query.
REQ-SYS-003	yes	yes	yes	Standard model-checking property.
REQ-SYS-004	yes	yes	yes	Encodes R12 with auxiliary state.
REQ-SYS-005	partial	not in CTL	yes	Requires LTL or `leadsto`.

Table 5: Per-requirement quality check.

5.3 Representation of Safety and Fairness

Two encodings were considered:

Direct location guard. Allow the Waiting→Working transition only on `release[id]?`. Structurally watertight, but produces no observable artefact.

Witness flag `wasReleased`. Per-Worker boolean set to true on `release[id]?` and reset to false on Working→Idle. The CTL query becomes `A[] Worker_i.Working imply Worker_i.wasReleased`.

We use encoding 2. It produces a meaningfully testable query: any future model edit that accidentally adds an unguarded Waiting→Working transition will be caught by the query, whereas encoding 1 hides such a regression.

Fairness (REQ-SYS-005). The textbook formalisation in LTL is

$$\Box\Diamond Requesting_i \;\Rightarrow\; \Box\Diamond Working_i,$$

which UPPAAL CTL cannot express directly — there is no $\Box\Diamond$ operator and no Büchi acceptance. The strongest expressible surrogate is `Worker_i.Requesting -> Worker_i.Working` together with reachability `E<> Worker_i.Working`. The full discussion is given in Section 7.4.

5.4 Formal vs. Informal Discharge

Category	**IDs**
Formally verifiable in UPPAAL	REQ-SYS-001..004, REQ-WRK-001..004, REQ-CTL-001..003, REQ-Q-001..003
Discharged automatically by formalism	REQ-CTL-004 (binary-channel atomicity)
Verified only as a weaker `leadsto`/reachability surrogate	REQ-SYS-005
Argued informally	ASM-005, the chosen upper bound in ASM-006, ASM-008 (initial state)

Table 6: Categorisation of requirements by verification approach.

5.5 Traceability Table

Maintaining a clear line of sight from the original problem statement to the final engineering decisions is a critical requirement of this task. Table 7 provides a comprehensive traceability matrix, linking each final SysML requirement and explicit assumption back to its natural language source while documenting the rationale for any changes or additions.

6 UPPAAL Model

6.1 Architecture

The model is composed of three networked timed automata:

$$\begin{array}{lcl} \text{Worker0} & \xrightarrow{\texttt{request[0]!}} & \text{Controller} \\ \text{Worker1} & \xrightarrow{\texttt{request[1]!}} & \text{Controller} \\ \text{Controller} & \xrightarrow{\texttt{release[i]!}} & \text{Worker}_i \end{array}$$

The controller owns the global FIFO queue. All inter-task communication uses per-id binary channel arrays `request[N]` and `release[N]`, which satisfies REQ-WRK-002 ("task-specific release") without resorting to broadcast channels.

ID	Requirement (short form)	Source	Change Rationale
ASM-001	$N = 2$.	Manual	Implicit in R1; made explicit.
ASM-002	$QSIZE = 4$, $QSIZE \geq N$.	Manual	R4 says "bounded" without a value.
ASM-003	One outstanding request per Worker.	Manual	Implicit in cycle; required for $count \leq N$.
ASM-004	Binary blocking channels.	Manual	Required for chosen sync semantics.
ASM-005	Scheduler fairness.	Manual	Without it R11 is unprovable.
ASM-006	Bounded Working duration.	Manual	Avoids Zeno; justifies clock.
ASM-007	Worker IDs $\in [0, N-1]$.	Manual	Required for channel arrays.
REQ-SYS-001	Exactly 2 Workers + 1 Controller.	LLM (R1)	Reformulated to SysML.
REQ-SYS-002	Worker ID type.	Manual	R1 does not type the ID.
REQ-WRK-001	`request!` before Working.	LLM (R2)	Made operational.
REQ-WRK-002	Wait for `release[id]?`.	LLM (R3)	A3 disambiguated.
REQ-WRK-003	Strict cycle, single outstanding.	Manual	Closes gap in R2/R3.
REQ-WRK-004	Bounded Working time.	Manual	Adds timing for Task 4 clock.
REQ-CTL-001	Enqueue atomically with `request?`.	LLM (R4)	Atomicity made explicit (A4).
REQ-CTL-002	Release `queue[head]` first.	LLM (R5)	Selection rule operationalised.
REQ-CTL-003	Dequeue atomic with release.	LLM/Manual (R6)	"After" replaced by "atomically with".
REQ-CTL-004	One release per step.	LLM (R7)	Recognised as guaranteed by ASM-004.
REQ-Q-001	$0 \leq count \leq QSIZE$.	LLM (R8)	Restated as invariant.
REQ-Q-002	Enqueue guard.	LLM (R8)	Operationalised.
REQ-Q-003	Dequeue guard.	LLM (R9)	Operationalised.
REQ-SYS-003	`A[] not deadlock`.	LLM (R10)	Stated as the exact CTL query.
REQ-SYS-004	`A[] Working ==> wasReleased`.	LLM (R12)	"Prior" disambiguated to "this cycle".
REQ-SYS-005	Strong fairness in LTL.	LLM (R11)	Translated; flagged as not directly checkable.

Table 7: Traceability of requirements to source and rationale.

6.2 Global Declarations

The exact declarations from the assignment specification are used verbatim, with two helper functions for atomic enqueue/dequeue.

```
const int N     = 2;        // number of worker tasks
const int QSIZE = 4;        // bounded FIFO capacity (QSIZE ≥ N)

typedef int[0, N-1]    task_t;      // valid worker IDs
typedef int[0, QSIZE]  qindex_t;    // valid head/tail/count values

chan request[N];                    // per-worker request channels
chan release[N];                    // per-worker release channels

task_t   queue[QSIZE];
qindex_t head  = 0;
qindex_t tail  = 0;
qindex_t count = 0;
task_t   selected = 0;              // last released worker (debug aid)

void enqueue(task_t id) {
    queue[tail] = id;
    tail        = (tail + 1) % QSIZE;
    count       = count + 1;
}

void dequeue() {
    selected = queue[head];
    head     = (head + 1) % QSIZE;
    count    = count - 1;
}
```

Listing 1: Global declarations and queue helpers.

The typedef `qindex_t = int[0,QSIZE]` doubles as a static type bound: UPPAAL flags any out-of-range assignment to `head`, `tail`, or `count` before model-checking begins.

6.3 Worker Template

The Worker template has four locations and a per-instance clock and witness flag.

Location	Role	Invariant
Idle	Initial / returned after a work cycle	—
Requesting	About to send the request	—
Waiting	Sent request, waiting for release	—
Working	Authorised to perform the task	$x \leq 10$

Table 8: Worker locations and invariants.

Local declarations:

```
clock x;                        // measures time spent in Working
bool  wasReleased = false;      // safety witness for REQ-SYS-004
```

Listing 2: Worker-template local declarations.

Transitions:

From → To	Sync	Guard	Update
Idle → Requesting	—	—	`x := 0`
Requesting → Waiting	`request[id]!`	—	—
Waiting → Working	`release[id]?`	—	`wasReleased := true; x := 0`
Working → Idle	—	$x \geq 2$	`wasReleased := false`

Table 9: Worker transitions.

A local clock per worker — rather than one global clock — avoids complex resets when both workers are simultaneously in Working. Bounding x from above ($x \leq 10$) prevents Zeno behaviour; bounding it from below ($x \geq 2$) makes elapsed time observable.

6.4 Controller Template

The controller is a single location Serving with two self-loop transitions: enqueue and release. This single-location design is a deliberate fix to a class of deadlocks observed in earlier multi-location prototypes; see Section 8, Iteration 2.

Branch	Guard / Sync / Update	Discharges
Enqueue (self-loop)	select `i : task_t` guard `count < QSIZE` sync `request[i]?` update `enqueue(i)`	REQ-CTL-001, REQ-Q-002
Release (self-loop)	guard `count > 0` sync `release[queue[head]]!` update `dequeue()`	REQ-CTL-002, REQ-CTL-003, REQ-Q-003

Table 10: Controller transitions in the final model.

Order of evaluation in `release[queue[head]]!`. UPPAAL evaluates the channel-index expression before running the update block. So `queue[head]` is read with the pre-state value of `head`, and only afterwards is `head` advanced and `count` decremented. This is exactly the FIFO release semantics demanded by REQ-CTL-002 and REQ-CTL-003.

The `select` clause. The enqueue self-loop uses `select i : task_t`. UPPAAL non-deterministically binds `i` to a value in $[0, N-1]$ at the moment of taking the transition, combined with the synchronisation `request[i]?`. The controller is therefore willing to receive a request from any worker; if multiple workers are simultaneously in Requesting, the selection order is non-deterministic, which is the correct semantic match for "concurrent requests".

6.5 System Declaration

```
Worker0 = Worker(0);
Worker1 = Worker(1);
Ctrl    = Controller();

system Worker0, Worker1, Ctrl;
```

Listing 3: Top-level system declaration.

Exactly two Worker instances and one Controller — this discharges REQ-SYS-001 statically.

ID	Discharge mechanism
REQ-SYS-001	`system Worker0, Worker1, Ctrl;` — exact cardinality, statically.
REQ-SYS-002	`task_t = int[0,N-1]` typedef; instances passed 0 and 1.
REQ-WRK-001	Requesting→Waiting is the only entry to Waiting; its sync is `request[id]!`.
REQ-WRK-002	Waiting→Working synchronises only on `release[id]?`.
REQ-WRK-003	Cyclic location structure; no transition skips ahead.
REQ-WRK-004	Invariant $x \leq 10$ and guard $x \geq 2$ on Working.
REQ-CTL-001	`enqueue(i)` as update of the `request[i]?` transition.
REQ-CTL-002	`release[queue[head]]!` always names the head element.
REQ-CTL-003	`dequeue()` is the update of the same release transition (atomicity).
REQ-CTL-004	UPPAAL binary channels — automatic.
REQ-Q-002	Guard `count < QSIZE`.
REQ-Q-003	Guard `count > 0`.
REQ-Q-001	Combined; also enforced statically by `qindex_t`.
REQ-SYS-003	Verified by `A[] not deadlock`.
REQ-SYS-004	`wasReleased` flag plus `A[] Working imply wasReleased`.
REQ-SYS-005	Surrogated by `E<> Working` and `Requesting -> Working`.

Table 11: How each requirement is discharged in the model.

6.6 Requirement Discharge Map

7 Verification Results

7.1 Query Set

The full query file submitted alongside this report is `04_verification_queries.q`. Its contents are summarised in Table 12.

7.2 Verification Matrix

The pass predictions stem from a manual symbolic walk-through; empirical confirmation with `verifyta` on the submitted XML is the next step in the engineering loop.

7.3 Why Each Property Holds (selected)

Deadlock freedom. A state is dead iff (i) no discrete transition is enabled and (ii) no clock progress is possible. In our model: Workers in Idle, Requesting, or Waiting have at least one outgoing transition that is eventually enabled. The controller is permanently in Serving; the enqueue branch is enabled whenever any worker is in Requesting (and `count < QSIZE` is always true since $count \leq N = 2 < 4$); the release branch is enabled whenever `count > 0`. Time can always progress in Working until the upper invariant.

Hence no state is dead.

Safety witness. By construction the `wasReleased` flag of `Worker_i` is exactly true on every state where the location is Working, and exactly false elsewhere. The implication `A[] Worker_i.Working imply Worker_i.wasReleased` therefore holds in every reachable state, by direct inspection of the transition labels.

Property / Intent	**Maps to**	**UPPAAL CTL Query**
1. Deadlock freedom	REQ-SYS-003 / R10	`A[] not deadlock`
2. No queue overflow	REQ-Q-002 / R8	`A[] count <= QSIZE`
3. No queue underflow	REQ-Q-003 / R9	`A[] count >= 0`
4. Safety per worker	REQ-SYS-004 / R12	`A[] Worker0.Working imply Worker0.wasReleased` `A[] Worker1.Working imply Worker1.wasReleased`
5. Reachability of work (sanity / weak liveness)	—	`E<> Worker0.Working` `E<> Worker1.Working`
6. Weak liveness surrogate for fairness	REQ-SYS-005 / R11	`Worker0.Requesting -> Worker0.Working` `Worker1.Requesting -> Worker1.Working`
7. Queue index sanity	REQ-Q-001	`A[] head < QSIZE` `A[] tail < QSIZE`
8. FIFO consistency (informal property)	—	`A[] count > 0 imply (queue[head] >= 0 and queue[head] <= N-1)`

Table 12: UPPAAL verification queries.

#	**Query**	**Property**	**Expected**	**Maps to**
1	`A[] not deadlock`	Deadlock freedom	pass	R10
2	`A[] count <= QSIZE`	No overflow	pass	R8
3	`A[] count >= 0`	No underflow	pass	R9
4	`A[] Worker0.Working imply Worker0.wasReleased`	Safety, W0	pass	R12
5	`A[] Worker1.Working imply Worker1.wasReleased`	Safety, W1	pass	R12
6	`E<> Worker0.Working`	Reachability	pass	R11 (weak)
7	`E<> Worker1.Working`	Reachability	pass	R11 (weak)
8	`Worker0.Requesting -> Worker0.Working`	`leadsto`	pass (under fairness)	R11 (partial)
9	`Worker1.Requesting -> Worker1.Working`	`leadsto`	pass (under fairness)	R11 (partial)
10	`A[] head < QSIZE`	Index sanity	pass	R8/R9
11	`A[] tail < QSIZE`	Index sanity	pass	R8/R9
12	`A[] count > 0 imply queue[head] in [0,N-1]`	Type sanity	pass	task_t

Table 13: Expected verification results.

7.4 Fairness Discussion

7.4.1 What R11 actually says

The textbook formalisation of R11 in LTL is strong fairness:

$$\Box\Diamond Requesting_i \;\Rightarrow\; \Box\Diamond Working_i,$$

i.e. if a worker is infinitely often in Requesting it is infinitely often in Working.

7.4.2 What UPPAAL CTL can express

Operator	Meaning	Strength	Used here
`A[]` φ	safety: φ holds on all paths in all states	strong	yes
`E<>` φ	reachability: $\exists$ a path that eventually reaches φ	weak	yes
`A<>` φ	inevitability: every path eventually reaches φ	strong, rare	no
`E[]` φ	possibility: $\exists$ a path on which φ always holds	weak	no
φ `->` ψ	every φ-state is followed in time by a ψ-state, on every path	medium	yes

Table 14: Summary of UPPAAL CTL operators relevant for R11.

UPPAAL CTL has no $\Box\Diamond$ operator and no Büchi acceptance — strong fairness in the LTL sense is structurally outside the logic.

7.4.3 Reachability is not the same as fairness

The temptation is to claim fairness because `E<> Worker0.Working` and `E<> Worker1.Working` both hold. They are not equivalent:

`E<> Worker0.Working` only states that some path exists on which Worker 0 reaches Working. It says nothing about every path, nothing about infinitely often, and nothing about every request being served.

A pathological scheduler that always lets Worker 1 go and never Worker 0 still satisfies `E<> Worker0.Working` because some path lets Worker 0 go. It clearly violates fairness.

7.4.4 Leadsto is closer but still weaker

`Worker0.Requesting -> Worker0.Working` is a universal property: every visit to Requesting is matched by a later visit to Working. It is the best UPPAAL CTL surrogate available, but it does not require infinitely many such matches; UPPAAL evaluates `->` under built-in fair-time-progress semantics, which catches the one-shot version of unfairness but not the infinite-tail Büchi version.

7.4.5 Standard workarounds for strong fairness

1. LTL via newer UPPAAL extensions (Stratego, TIGA): re-encode the model with one clock and the property as a Büchi automaton.
2. Fairness-witness automaton: add an extra automaton that monitors Requesting/Working for each worker, has its own unfair location, and prove `A[] not unfair`.
3. Bounded-horizon reduction: prove `A[] (Requesting imply Working_within_T)` for some chosen T.

7.4.6 Conclusion on fairness

A strict mathematical proof of strong fairness is not delivered by this UPPAAL model; UPPAAL CTL is structurally too weak. What we do prove is the `leadsto` surrogate `Requesting -> Working` for each worker, plus reachability `E<> Working`. The strong-fairness claim ultimately rests on ASM-005 — scheduler/environment fairness — and on the syntactic symmetry of the controller's two release branches w.r.t. both workers.

8 Iterations and Changes

A structured dry-run of the symbolic state space was performed at each iteration, looking for dead states, queue overflow/underflow, transitions into Working without `wasReleased=true`, and starvation paths. Five self-correction iterations were applied before the model stabilised.

8.1 Iteration 0 — Naive Initial Sketch

The first sketch used a multi-location controller (Idle→Enqueueing→Idle, Idle→Releasing→Idle). Two flaws were detected by the dry-run:

Flaw 1 — non-atomic dequeue. If `head` and `count` are updated before the synchronisation, the FIFO order is broken: on certain schedules the worker at the original head can starve.

Flaw 2 — committed-location deadlock. A committed Releasing location with sync `release[queue[head]]!` cannot be left until the matching worker is in Waiting. If that worker is not yet there (it has not yet executed its `request[id]!`), the controller is stuck in the committed location and the system deadlocks.

8.2 Iteration 1 — Atomic Dequeue

Move the index/`count` updates into the same update block as the synchronisation:

```
// transition: release[queue[head]]!   (sync)
// update    : selected = queue[head];
//             head     = (head + 1) % QSIZE;
//             count    = count - 1;
```

The expression `release[queue[head]]` is evaluated using the pre-state of `head`, so the FIFO order is preserved.

8.3 Iteration 2 — Single-Location Controller

Collapse Idle/Enqueueing/Releasing into a single Serving location with two self-loops (Section 6.4). Effects:

No transient location can lock the controller. No committed location is needed. The state space contributed by the controller drops to one symbolic state per $(count, head, tail, queue)$ tuple.

8.4 Iteration 3 — Local, Resettable `wasReleased`

The first attempt used a global `bool wasReleased[N]`. This works formally but breaks template encapsulation. Moving `wasReleased` into the Worker template gives the elegant query form `A[] Worker0.Working imply Worker0.wasReleased`.

A second sub-issue: an early version did not reset `wasReleased := false` on Working→Idle. That left the flag at true forever after the first cycle, masking subsequent violations. The reset was added.

8.5 Iteration 4 — Clock and Bounded Working

The original sketch had no clock at all, violating the assignment requirement "at least one clock for timing behaviour" and admitting Zeno traces. Added a local clock `x` per Worker, an invariant `x <= 10` on Working, a guard `x >= 2` on Working→Idle, and `x := 0` on both Idle→Requesting and Waiting→Working.

8.6 Iteration 5 — Range-Typed `qindex_t`

The first sketch used `int head, tail, count`. Switching to `typedef int[0,QSIZE] qindex_t` adds a static bound check on top of the dynamic guards: any update that would push the indices outside their range is detected by the UPPAAL type system before model-checking begins.

8.7 Summary

It.	Symptom	Fix	Affected req.
0	Multi-location ctrl, non-atomic dequeue	identified	REQ-CTL-002,003
1	FIFO order broken (Flaw 1)	atomic dequeue inside update	REQ-CTL-002,003
2	Committed-location deadlock (Flaw 2)	single Serving location	REQ-SYS-003
3	`wasReleased` global; never reset	local + reset on exit Working	REQ-SYS-004
4	No clock; Zeno admissible	local clock with invariant + guard	REQ-WRK-004, ASM-006
5	Indices untyped	typedef `qindex_t = int[0,QSIZE]`	REQ-Q-001

Table 15: Iteration log summary.

After Iteration 5 every dry-run scenario completes without deadlock or property violation. The single residual open item is the strict strong-fairness property (R11), which is documented as a CTL-formalism limitation rather than a model flaw.

9 FreeRTOS Implementation (Outlook)

Task 7 is an extra task and is not implemented in this submission. This section sketches the planned mapping from the UPPAAL model to FreeRTOS primitives, so that the design crosses the abstraction boundary cleanly.

9.1 Mapping

To transition from the formally verified model to a concrete software implementation, the abstract UPPAAL constructs must be mapped to their corresponding FreeRTOS primitives. Table 16 outlines this planned mapping, detailing how the modelled automata, synchronisation channels, and time constraints will be realised using FreeRTOS tasks, queues, direct-to-task notifications, and blocking delay functions.

9.2 Pseudo-code Skeleton

Listing 4: Worker and controller skeletons (planned).

UPPAAL artefact	FreeRTOS primitive
Worker template (instantiated 2×)	Two FreeRTOS tasks created with `xTaskCreate`, distinct priorities possible.
Controller template	One FreeRTOS task at higher priority than the workers.
`request[i]` channel array	A single FreeRTOS queue `xRequestQueue` of capacity `QSIZE` carrying worker IDs (FIFO is built-in).
`release[i]` channel array	Per-worker direct-to-task notifications (`xTaskNotifyGive` / `ulTaskNotifyTake`) — one notification slot per worker.
`wasReleased` witness	Implemented in worker code as a local boolean reset on Idle re-entry.
`x` clock + `x>=2`/`x<=10`	FreeRTOS `vTaskDelay(pdMS_TO_TICKS(...))` bounded by lower and upper bounds.

Table 16: Planned UPPAAL-to-FreeRTOS mapping.

```c
QueueHandle_t  xRequestQueue;            /* size = QSIZE, stores worker IDs */
TaskHandle_t   xWorkerHandles[N];

static void Worker(void *pvParameters) {
    UBaseType_t id = (UBaseType_t)(uintptr_t)pvParameters;
    bool wasReleased;
    for (;;) {
        wasReleased = false;
        /* Idle -> Requesting */
        xQueueSend(xRequestQueue, &id, portMAX_DELAY);      /* request[id]! */
        /* Waiting -> Working */
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);            /* release[id]? */
        wasReleased = true;
        configASSERT(wasReleased);                          /* REQ-SYS-004 */
        /* perform work */
        vTaskDelay(pdMS_TO_TICKS(2000));                    /* x >= 2 */
        /* Working -> Idle */
    }
}

static void Controller(void *pvParameters) {
    UBaseType_t selected;
    for (;;) {
        if (xQueueReceive(xRequestQueue, &selected, portMAX_DELAY) == pdPASS) {
            xTaskNotifyGive(xWorkerHandles[selected]);      /* release[selected]! */
        }
    }
}
```

The FreeRTOS queue gives FIFO semantics natively, so manual head/tail management is not needed at the implementation level — but the UPPAAL model keeps the explicit array because that is the property under verification.

9.3 Open Implementation Questions

Priority assignment between the two workers and the controller (if equal, the FreeRTOS scheduler enforces a fair round-robin — this is the FreeRTOS-side instance of ASM-005).

Whether `xQueueReceive` with timeout, instead of `portMAX_DELAY`, is needed to detect a frozen queue.

How to detect a missed notification (`ulTaskNotifyTake` returning 0) and recover.

10 Comparison and Critical Reflection

10.1 Artefact Comparison

Artefact	**Key change relative to predecessor**	**Origin**
Initial NL problem statement	—	assignment
LLM-generated requirements	R1–R12 rewritten in SysML form; ambiguities mostly identified	LLM
Final manually validated requirements	7 explicit ASM-items; REQ-CTL-003 atomicity strengthened; REQ-WRK-003/REQ-WRK-004 added	engineer
LLM-generated UPPAAL model	multi-location controller, no clock, global `wasReleased`, untyped indices	LLM
Final verified UPPAAL model	single-location controller, local clock + invariants, local `wasReleased`, typed `qindex_t`	engineer (5 iterations, see Section 8)

Table 17: Engineering artefact diff.

10.2 Where the LLM Helped

Boilerplate translation from natural language to SysML form ("shall <verb> <object>"). This freed the engineer from mechanical work.

First-pass detection of obvious ambiguities (e.g., A1, A3) was adequate.

Drafting the global declaration block exactly matching the assignment specification.

Generating the verification query set in syntactically correct UPPAAL CTL.

10.3 Where the LLM Failed or Introduced Risk

Initial controller design preferred multi-location pipelines with committed locations — introducing the deadlock of Iteration 2.

Initial `wasReleased` was global and never reset, masking subsequent safety violations.

No clock was included unprompted, although the assignment explicitly required one.

Strong-fairness $\Box\Diamond \Rightarrow \Box\Diamond$ was naively translated as `E<> Working`, which is a much weaker property. The engineer caught this and added the `leadsto` surrogate plus explicit limitation discussion.

Ambiguity A4 ("after selecting") was not flagged by the first-pass LLM output and had to be added manually.

10.4 Manual Engineering Decisions

1. Choosing $QSIZE = 4$ as a defensive bound while $count \leq N = 2$.
2. Collapsing the controller to a single Serving location.
3. Making `wasReleased` local and resettable.
4. Picking $x \in [2, 10]$ as the Working duration — a real-time choice that affects model-checking but not satisfiability of the safety properties.
5. Replacing the impossible-in-CTL strong-fairness query with the `leadsto` pair.

10.5 Verified vs. Argued Requirements

Verified by UPPAAL CTL	REQ-SYS-001..004, REQ-WRK-001..004, REQ-CTL-001..003, REQ-Q-001..003
Discharged by the formalism	REQ-CTL-004 (binary-channel atomicity)
Verified weakly only (surrogate)	REQ-SYS-005 (fairness, via `E<>` and `leadsto`)
Argued informally	ASM-005, the choice of upper bound in ASM-006, initial-state assumption ASM-008

Table 18: Verification status of each requirement.

References

[1] FreeRTOS Documentation. *Task scheduling.* https://freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/04-Task-scheduling

[2] FreeRTOS Documentation. *Tasks and task priorities.* https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/03-Task-priorities

[3] G. Behrmann, A. David, K. G. Larsen. *A Tutorial on UPPAAL.* https://uppaal.org/texts/21-tutorial.pdf

[4] No Magic Documentation. *Requirement Diagram.* https://docs.nomagic.com/spaces/SYSMLP2024x/pages/136725491/Requirement+Diagram

[5] *Large Language Models for Requirements Engineering: A Systematic Literature Review.* https://arxiv.org/html/2509.11446v1

[6] S. Henkler and A. Rettberg. *Engineering Task: From Requirements to UPPAAL and FreeRTOS*. Course handout, Real-Time Systems & HW/SW Co-Design.